

Reviewer Pack — Minimal Local Cohomology Rank Correlation with Communication...

Entry 39051a5999ec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 03:20:35 UTC

Minimal Local Cohomology Rank Correlation with Communication Complexity Growth for Planar Graphs

Entry ID: 39051a5999ec **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 03:20:35 UTC

1. Conjecture

Field A (mathematical branch): Local Cohomology Theory **Field B** (complexity object): Communication Complexity

Statement:

For any given planar graph G , the minimal local cohomology rank ($\text{lcr}(G)$) of its associated simplicial complex is linearly correlated with the growth rate of its communication complexity, such that $\text{lcr}(G) = \Theta(\text{growth_rate}(G))$.

Rationale (proposer's reasoning):

Local cohomology theory provides a measure of the complexity of algebraic structures, which may capture the inherent structure in the graph's communication process. A higher local cohomology rank suggests more complex interactions within the graph, potentially leading to increased communication complexity.

Taxonomy category: LOCAL_COHOMOLOGY_X_COMMUNICATION_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1adfabfcf61de04c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a planar graph G , if the correlation coefficient between $\text{lcr}(G)$ and $\text{growth_rate}(G)$ is ≥ 0.8 across 30 random seeds, then support the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local Cohomology Rank AND Communication Complexity IN title OR abstract - $\text{lcr}(G)$ AND $\theta(\text{growth_rate}(G))$ IN title OR abstract FOR planar graphs - Local Cohomology Theory AND Communication Complexity Growth IN title OR abstract FOR simplicial complexes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_planar(graph):
        # Implement a planarity test (e.g., Kuratowski's theorem check)
        # This is a simplified version and may not work for all cases.
        if len(graph) > 5:
            return False
        return True

    def local_cohomology_rank(simplicial_complex):
        # Placeholder function to compute local cohomology rank
        # This should be replaced with actual computation.
        return random.randint(1, 10)

    def communication_complexity(graph):
        # Placeholder function to compute communication complexity
        # This should be replaced with actual computation.
        return len(graph) ** 2

    instances_tested = 0
    total_lcr = 0
    total_growth_rate = 0
    n_max = 1

    for n in [5, 10, 15, 20, 30, 40]:
        if n > n_max:
            n_max = n
```

```

for _ in range(5): # Aim for at least 30 instances per seed
    graph = {i: set() for i in range(n)}
    for _ in range(n * (n - 1) // 2):
        u, v = random.sample(range(n), 2)
        if u != v and u not in graph[v]:
            graph[u].add(v)
            graph[v].add(u)

    if is_planar(graph):
        instances_tested += 1
        lcr = local_cohomology_rank(graph)
        growth_rate = communication_complexity(graph)
        total_lcr += lcr
        total_growth_rate += growth_rate

avg_lcr = total_lcr / instances_tested if instances_tested > 0 else 0
avg_growth_rate = total_growth_rate / instances_tested if instances_tested > 0 else 0

correlation_coefficient = (instances_tested * avg_lcr * avg_growth_rate -
    sum(lcr * growth_rate for lcr, growth_rate in zip([avg_lcr] * instances_tested, [avg_growth_rate] * instances_tested)) /
    math.sqrt(instances_tested * sum((lcr - avg_lcr) ** 2 for lcr in [avg_lcr] * instances_tested) *
    instances_tested * sum((growth_rate - avg_growth_rate) ** 2 for growth_rate in [avg_growth_rate] * instances_tested)))

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": "" if correlation_coefficient >= 0.8 else "correlation_coefficient < 0.8"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e79da3b4.py", line 84, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e79da3b4.py", line 65, in run_trial
    correlation_coefficient = (instances_tested * avg_lcr * avg_growth_rate -
                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution due to a division by zero error, which prevented the calculation of the correlation coefficient and its comparison against the pre-registered support condition. | next: Investigate the cause of the division by zero error in the test code. Once resolved, re-run the test with a larger sample size or different seeds to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24586
2	preregistration	ollama_remote	glm4:latest	0	0	12155
3	novelty	ollama_remote	glm4:latest	0	0	9573
4	novelty	ollama_remote	glm4:latest	0	0	13084
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18736
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13885
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19143
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24899
9	judge	ollama_remote	glm4:latest	0	0	10466

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146527 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/39051a5999ec.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/39051a5999ec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/39051a5999ec.tar.gz` (if generated)